

Project 2:Diabetes Prediction

importing the Dependencies

```
In [1]: import numpy as np
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn import svm
from sklearn.metrics import accuracy_score
```

Data Collection and Analysis

PLMA Diabetes Dataset

```
In [2]: #Loading the diabetes dataset
diabetes_dataset=pd.read_csv(r"D:\Projects\archive (1)\diabetes_prediction_data.csv")
```

```
In [3]: diabetes_dataset.head()
```

```
Out[3]:
```

	gender	age	hypertension	heart_disease	smoking_history	bmi	HbA1c_level	blood_sugar
0	Female	80.0	0	1	never	25.19	6.6	1
1	Female	54.0	0	0	No Info	27.32	6.6	1
2	Male	28.0	0	0	never	27.32	5.7	1
3	Female	36.0	0	0	current	23.45	5.0	1
4	Male	76.0	1	1	current	20.14	4.8	1



```
In [4]: pd.read_csv?
```

Signature:

```
pd.read_csv(
    filepath_or_buffer: 'FilePath | ReadCsvBuffer[bytes] | ReadCsvBuffer[str]',
    *,
    sep: 'str | None | lib.NoDefault' = <no_default>,
    delimiter: 'str | None | lib.NoDefault' = None,
    header: "int | Sequence[int] | None | Literal['infer']" = 'infer',
    names: 'Sequence[Hashable] | None | lib.NoDefault' = <no_default>,
    index_col: 'IndexLabel | Literal[False] | None' = None,
    usecols: 'UsecolsArgType' = None,
    dtype: 'DtypeArg | None' = None,
    engine: 'CSVEngine | None' = None,
    converters: 'Mapping[Hashable, Callable] | None' = None,
    true_values: 'list | None' = None,
    false_values: 'list | None' = None,
    skipinitialspace: 'bool' = False,
    skiprows: 'list[int] | int | Callable[[Hashable], bool] | None' = None,
    skipfooter: 'int' = 0,
    nrows: 'int | None' = None,
    na_values: 'Hashable | Iterable[Hashable] | Mapping[Hashable, Iterable[Hashable]] | None' = None,
    keep_default_na: 'bool' = True,
    na_filter: 'bool' = True,
    verbose: 'bool | lib.NoDefault' = <no_default>,
    skip_blank_lines: 'bool' = True,
    parse_dates: 'bool | Sequence[Hashable] | None' = None,
    infer_datetime_format: 'bool | lib.NoDefault' = <no_default>,
    keep_date_col: 'bool | lib.NoDefault' = <no_default>,
    date_parser: 'Callable | lib.NoDefault' = <no_default>,
    date_format: 'str | dict[Hashable, str] | None' = None,
    dayfirst: 'bool' = False,
    cache_dates: 'bool' = True,
    iterator: 'bool' = False,
    chunksize: 'int | None' = None,
    compression: 'CompressionOptions' = 'infer',
    thousands: 'str | None' = None,
    decimal: 'str' = '.',
    lineterminator: 'str | None' = None,
    quotechar: 'str' = '"',
    quoting: 'int' = 0,
    doublequote: 'bool' = True,
    escapechar: 'str | None' = None,
    comment: 'str | None' = None,
    encoding: 'str | None' = None,
    encoding_errors: 'str | None' = 'strict',
    dialect: 'str | csv.Dialect | None' = None,
    on_bad_lines: 'str' = 'error',
    delim_whitespace: 'bool | lib.NoDefault' = <no_default>,
    low_memory: 'bool' = True,
    memory_map: 'bool' = False,
    float_precision: "Literal['high', 'legacy'] | None" = None,
    storage_options: 'StorageOptions | None' = None,
    dtype_backend: 'DtypeBackend | lib.NoDefault' = <no_default>,
) -> 'DataFrame | TextFileReader'
```

Docstring:

Read a comma-separated values (csv) file into DataFrame.

Also supports optionally iterating or breaking of the file into chunks.

Additional help can be found in the online docs for
 IO Tools <https://pandas.pydata.org/pandas-docs/stable/user_guide/io.html>`_.

Parameters

`filepath_or_buffer` : str, path object or file-like object

Any valid string path is acceptable. The string could be a URL. Valid URL schemes include http, ftp, s3, gs, and file. For file URLs, a host is expected. A local file could be: `file://localhost/path/to/table.csv`.

If you want to pass in a path object, pandas accepts any `os.PathLike`.

By file-like object, we refer to objects with a `read()` method, such as a file handle (e.g. via builtin `open` function) or `StringIO`.

`sep` : str, default ',',

Character or regex pattern to treat as the delimiter. If `sep=None`, the C engine cannot automatically detect the separator, but the Python parsing engine can, meaning the latter will be used and automatically detect the separator from only the first valid row of the file by Python's builtin sniffer tool, `csv.Sniffer`.

In addition, separators longer than 1 character and different from `'\s+'` will be interpreted as regular expressions and will also force the use of the Python parsing engine. Note that regex delimiters are prone to ignoring quoted data. Regex example: `'\r\t'`.

`delimiter` : str, optional

Alias for `sep`.

`header` : int, Sequence of int, 'infer' or None, default 'infer'

Row number(s) containing column labels and marking the start of the data (zero-indexed). Default behavior is to infer the column names: if no `names`

are passed the behavior is identical to `header=0` and column names are inferred from the first line of the file, if column names are passed explicitly to `names` then the behavior is identical to `header=None`. Explicitly pass `header=0` to be able to replace existing names. The header can be a list of integers that specify row locations for a `:class:`~pandas.MultiIndex`` on the columns e.g. `[0, 1, 3]`. Intervening rows that are not specified will be skipped (e.g. 2 in this example is skipped). Note that this parameter ignores commented lines and empty lines if `skip_blank_lines=True`, so `header=0` denotes the first line of data rather than the first line of the file.

`names` : Sequence of Hashable, optional

Sequence of column labels to apply. If the file contains a header row, then you should explicitly pass `header=0` to override the column names. Duplicates in this list are not allowed.

`index_col` : Hashable, Sequence of Hashable or False, optional

Column(s) to use as row label(s), denoted either by column labels or column indices. If a sequence of labels or indices is given, `:class:`~pandas.MultiIndex`` will be formed for the row labels.

Note: `index_col=False` can be used to force pandas to *not* use the first column as the index, e.g., when you have a malformed file with delimiters at the end of each line.

`usecols` : Sequence of Hashable or Callable, optional

Subset of columns to select, denoted either by column labels or column indices.

If list-like, all elements must either be positional (i.e. integer indices into the document columns) or strings that correspond to column names provided either by the user in `names` or

inferred from the document header row(s). If ``names`` are given, the document header row(s) are not taken into account. For example, a valid list-like ``usecols`` parameter would be ``[0, 1, 2]`` or ``['foo', 'bar', 'baz']``. Element order is ignored, so ``usecols=[0, 1]`` is the same as ``[1, 0]``. To instantiate a `pandas.DataFrame` from ``data`` with element order preserved use `pd.read_csv(data, usecols=['foo', 'bar'])[['foo', 'bar']]` for columns in ``['foo', 'bar']`` order or `pd.read_csv(data, usecols=['foo', 'bar'])[['bar', 'foo']]` for ``['bar', 'foo']`` order.

If callable, the callable function will be evaluated against the column names, returning names where the callable function evaluates to ``True``. An example of a valid callable argument would be `lambda x: x.upper()` in `['AAA', 'BBB', 'DDD']`. Using this parameter results in much faster parsing time and lower memory usage.

`dtype` : dtype or dict of {Hashable : dtype}, optional
Data type(s) to apply to either the whole dataset or individual columns. E.g., `{'a': np.float64, 'b': np.int32, 'c': 'Int64'}`. Use ``str`` or ``object`` together with suitable ``na\_values`` settings to preserve and not interpret ``dtype``. If ``converters`` are specified, they will be applied INSTEAD of ``dtype`` conversion.

.. versionadded:: 1.5.0

Support for ``defaultdict`` was added. Specify a ``defaultdict`` as input where the default determines the ``dtype`` of the columns which are not explicitly listed.

`engine` : {'c', 'python', 'pyarrow'}, optional
Parser engine to use. The C and pyarrow engines are faster, while the python engine is currently more feature-complete. Multithreading is currently only supported by the pyarrow engine.

.. versionadded:: 1.4.0

The 'pyarrow' engine was added as an *experimental* engine, and some features are unsupported, or may not work correctly, with this engine.

`converters` : dict of {Hashable : Callable}, optional
Functions for converting values in specified columns. Keys can either be column labels or column indices.

`true_values` : list, optional
Values to consider as ``True`` in addition to case-insensitive variants of 'True'.

`false_values` : list, optional
Values to consider as ``False`` in addition to case-insensitive variants of 'False'.

`skipinitialspace` : bool, default False
Skip spaces after delimiter.

`skiprows` : int, list of int or Callable, optional
Line numbers to skip (0-indexed) or number of lines to skip (``int``) at the start of the file.

If callable, the callable function will be evaluated against the row indices, returning ``True`` if the row should be skipped and ``False`` otherwise.

ise.

An example of a valid callable argument would be `lambda x: x in [0, 2]`.

`skipfooter` : int, default 0

Number of lines at bottom of file to skip (Unsupported with `engine='c'`).

`nrows` : int, optional

Number of rows of file to read. Useful for reading pieces of large files.

`na_values` : Hashable, Iterable of Hashable or dict of {Hashable : Iterable}, optional

Additional strings to recognize as `NA`/`NaN`. If `dict` passed, specific

per-column `NA` values. By default the following values are interpreted as `NaN`: " ", "#N/A", "#N/A N/A", "#NA", "-1.#IND", "-1.#QNAN", "-NaN", "-nan",

"1.#IND", "1.#QNAN", "<NA>", "N/A", "NA", "NULL", "NaN", "None", "n/a", "nan", "null".

`keep_default_na` : bool, default True

Whether or not to include the default `NaN` values when parsing the data.

Depending on whether `na_values` is passed in, the behavior is as follows:

- * If `keep_default_na` is `True`, and `na_values` are specified, `na_values`

is appended to the default `NaN` values used for parsing.

- * If `keep_default_na` is `True`, and `na_values` are not specified, only

the default `NaN` values are used for parsing.

- * If `keep_default_na` is `False`, and `na_values` are specified, only the `NaN` values specified `na_values` are used for parsing.

- * If `keep_default_na` is `False`, and `na_values` are not specified, no

strings will be parsed as `NaN`.

Note that if `na_filter` is passed in as `False`, the `keep_default_na` and

`na_values` parameters will be ignored.

`na_filter` : bool, default True

Detect missing value markers (empty strings and the value of `na_values`).

In

data without any `NA` values, passing `na_filter=False` can improve the performance of reading a large file.

`verbose` : bool, default False

Indicate number of `NA` values placed in non-numeric columns.

.. deprecated:: 2.2.0

`skip_blank_lines` : bool, default True

If `True`, skip over blank lines rather than interpreting as `NaN` values.

`parse_dates` : bool, list of Hashable, list of lists or dict of {Hashable : list}, default False

The behavior is as follows:

- * `bool`. If `True` -> try parsing the index. Note: Automatically set to `True` if `date_format` or `date_parser` arguments have been passed.
- * `list` of `int` or names. e.g. If `[1, 2, 3]` -> try parsing columns 1, 2, 3

each as a separate date column.

- * `list` of `list`. e.g. If `[[1, 3]]` -> combine columns 1 and 3 and

parse as a single date column. Values are joined with a space before parsing.

- * `dict`, e.g. `{'foo' : [1, 3]}` -> parse columns 1, 3 as date and call

result 'foo'. Values are joined with a space before parsing.

If a column or index cannot be represented as an array of ``datetime``, say because of an unparsable value or a mixture of timezones, the column or index will be returned unaltered as an ``object`` data type. For non-standard ``datetime`` parsing, use :func:`~pandas.to\_datetime` after :func:`~pandas.read\_csv`.

Note: A fast-path exists for iso8601-formatted dates.

`infer_datetime_format` : bool, default False

If ``True`` and ``parse\_dates`` is enabled, pandas will attempt to infer the format of the ``datetime`` strings in the columns, and if it can be inferred, switch to a faster method of parsing them. In some cases this can increase the parsing speed by 5-10x.

.. deprecated:: 2.0.0

A strict version of this argument is now the default, passing it has no effect.

`keep_date_col` : bool, default False

If ``True`` and ``parse\_dates`` specifies combining multiple columns then keep the original columns.

`date_parser` : Callable, optional

Function to use for converting a sequence of string columns to an array of ``datetime`` instances. The default uses ``dateutil.parser.parser`` to do the conversion. pandas will try to call ``date\_parser`` in three different ways, advancing to the next if an exception occurs: 1) Pass one or more arrays (as defined by ``parse\_dates``) as arguments; 2) concatenate (row-wise) the string values from the columns defined by ``parse\_dates`` into a single array and pass that; and 3) call ``date\_parser`` once for each row using one or more strings (corresponding to the columns defined by ``parse\_dates``) as arguments.

.. deprecated:: 2.0.0

Use ``date\_format`` instead, or read in as ``object`` and then apply :func:`~pandas.to\_datetime` as-needed.

`date_format` : str or dict of column -> format, optional

Format to use for parsing dates when used in conjunction with ``parse\_dates``.

The strftime to parse time, e.g. :const:`~"%d/%m/%Y"`. See `strftime` documentation

<<https://docs.python.org/3/library/datetime.html>

#strftime-and-strptime-behavior> for more information on choices, though note that :const:`~"%f"`` will parse all the way up to nanoseconds.

You can also pass:

- "ISO8601", to parse any `ISO8601` <https://en.wikipedia.org/wiki/ISO_8601> time string (not necessarily in exactly the same format);
- "mixed", to infer the format for each element individually. This is risky, and you should probably use it along with `dayfirst`.

.. versionadded:: 2.0.0

`dayfirst` : bool, default False

DD/MM format dates, international and European format.

`cache_dates` : bool, default True

If ``True``, use a cache of unique, converted dates to apply the ``datetime`` conversion. May produce significant speed-up when parsing duplicate date strings, especially ones with timezone offsets.

`iterator` : bool, default False

Return ``TextFileReader`` object for iteration or getting chunks with
 ``get\_chunk()``.

chunks : int, optional
 Number of lines to read from the file per chunk. Passing a value will cause the
 function to return a ``TextFileReader`` object for iteration.
 See the `IO Tools docs`
<https://pandas.pydata.org/pandas-docs/stable/io.html#io-chunking> for
 more information on ``iterator`` and ``chunks``.

compression : str or dict, default 'infer'
 For on-the-fly decompression of on-disk data. If 'infer' and 'filepath_or_buffer' is
 path-like, then detect compression from the following extensions: '.gz',
 '.bz2', '.zip', '.xz', '.zst', '.tar', '.tar.gz', '.tar.xz' or '.tar.bz2'
 (otherwise no compression).
 If using 'zip' or 'tar', the ZIP file must contain only one data file to be read in.
 Set to ``None`` for no decompression.
 Can also be a dict with key ``'method'`` set
 to one of {``'zip'``, ``'gzip'``, ``'bz2'``, ``'zstd'``, ``'xz'``, ``'tar'``}
 and
 other key-value pairs are forwarded to
 ``zipfile.ZipFile``, ``gzip.GzipFile``,
 ``bz2.BZ2File``, ``zstandard.ZstdDecompressor``, ``lzma.LZMAFile`` or
 ``tarfile.TarFile``, respectively.
 As an example, the following could be passed for Zstandard decompression using a
 custom compression dictionary:
 ``compression={'method': 'zstd', 'dict\_data': my\_compression\_dict}``.

.. versionadded:: 1.5.0
 Added support for ``.tar`` files.

.. versionchanged:: 1.4.0 Zstandard support.

thousands : str (length 1), optional
 Character acting as the thousands separator in numerical values.

decimal : str (length 1), default '.'
 Character to recognize as decimal point (e.g., use ',' for European data).

lineterminator : str (length 1), optional
 Character used to denote a line break. Only valid with C parser.

quotechar : str (length 1), optional
 Character used to denote the start and end of a quoted item. Quoted
 items can include the ``delimiter`` and it will be ignored.

quoting : {0 or csv.QUOTE_MINIMAL, 1 or csv.QUOTE_ALL, 2 or csv.QUOTE_NONNUMERIC,
 3 or csv.QUOTE_NONE}, default csv.QUOTE_MINIMAL
 Control field quoting behavior per ``csv.QUOTE\_\*`` constants. Default is
 ``csv.QUOTE\_MINIMAL`` (i.e., 0) which implies that only fields containing special
 characters are quoted (e.g., characters defined in ``quotechar``, ``delimiter``
 or ``lineterminator``).

doublequote : bool, default True
 When ``quotechar`` is specified and ``quoting`` is not ``QUOTE\_NONE``, indicate
 whether or not to interpret two consecutive ``quotechar`` elements INSIDE a
 field as a single ``quotechar`` element.

escapechar : str (length 1), optional
 Character used to escape other characters.

```

comment : str (length 1), optional
    Character indicating that the remainder of line should not be parsed.
    If found at the beginning
    of a line, the line will be ignored altogether. This parameter must be a
    single character. Like empty lines (as long as ``skip_blank_lines=True``),
    fully commented lines are ignored by the parameter ``header`` but not by
    ``skiprows``. For example, if ``comment='#'``, parsing
    ``#empty\na,b,c\n1,2,3`` with ``header=0`` will result in ``'a,b,c'`` being
    treated as the header.
encoding : str, optional, default 'utf-8'
    Encoding to use for UTF when reading/writing (ex. ``'utf-8'``). `List of Python
on
    standard encodings
    <https://docs.python.org/3/library/codecs.html#standard-encodings>`_ .

encoding_errors : str, optional, default 'strict'
    How encoding errors are treated. `List of possible values
    <https://docs.python.org/3/library/codecs.html#error-handlers>`_ .

.. versionadded:: 1.3.0

dialect : str or csv.Dialect, optional
    If provided, this parameter will override values (default or not) for the
    following parameters: ``delimiter``, ``doublequote``, ``escapechar``,
    ``skipinitialspace``, ``quotechar``, and ``quoting``. If it is necessary to
    override values, a ``ParserWarning`` will be issued. See ``csv.Dialect``
    documentation for more details.
on_bad_lines : {'error', 'warn', 'skip'} or Callable, default 'error'
    Specifies what to do upon encountering a bad line (a line with too many field
s).
    Allowed values are :

    - ``'error'``, raise an Exception when a bad line is encountered.
    - ``'warn'``, raise a warning when a bad line is encountered and skip that li
ne.
    - ``'skip'``, skip bad lines without raising or warning when they are encount
ered.

.. versionadded:: 1.3.0

.. versionadded:: 1.4.0

    - Callable, function with signature
      `` (bad_line: list[str]) -> list[str] | None`` that will process a singl
e
      bad line. ``bad_line`` is a list of strings split by the ``sep``.
      If the function returns ``None``, the bad line will be ignored.
      If the function returns a new ``list`` of strings with more elements th
an
      expected, a ``ParserWarning`` will be emitted while dropping extra elem
ents.
      Only supported when ``engine='python'``

.. versionchanged:: 2.2.0

    - Callable, function with signature
      as described in `pyarrow documentation
    <https://arrow.apache.org/docs/python/generated/pyarrow.csv.ParseOptions
s.html
      #pyarrow.csv.ParseOptions.invalid_row_handler>`_ when ``engine='pyarro

```


w'''

`delim_whitespace` : bool, default False

Specifies whether or not whitespace (e.g. `''' '''` or `'''\t'''`) will be used as the `sep` delimiter. Equivalent to setting `sep='\s+'`. If this option

is set to `True`, nothing should be passed in for the `delimiter` parameter.

.. deprecated:: 2.2.0

Use `sep="\s+"` instead.

`low_memory` : bool, default True

Internally process the file in chunks, resulting in lower memory use while parsing, but possibly mixed type inference. To ensure no mixed types either set `False`, or specify the type with the `dtype` parameter. Note that the entire file is read into a single `pandas.DataFrame` regardless, use the `chunksize` or `iterator` parameter to return the data in

chunks. (Only valid with C parser).

`memory_map` : bool, default False

If a filepath is provided for `filepath_or_buffer`, map the file object directly onto memory and access the data directly from there. Using this option can improve performance because there is no longer any I/O overhead.

`float_precision` : {'high', 'legacy', 'round_trip'}, optional

Specifies which converter the C engine should use for floating-point values. The options are `None` or `'high'` for the ordinary converter, `'legacy'` for the original lower precision pandas converter, and `'round_trip'` for the round-trip converter.

`storage_options` : dict, optional

Extra options that make sense for a particular storage connection, e.g. host, port, username, password, etc. For HTTP(S) URLs the key-value pairs are forwarded to `urllib.request.Request` as header options. For other URLs (e.g. starting with `s3://`, and `gcs://`) the key-value pairs are forwarded to `fsspec.open`. Please see `fsspec` and `urllib` for more details, and for more examples on storage options refer [here](https://pandas.pydata.org/docs/user_guide/io.html?highlight=storage_options#reading-writing-remote-files) `<https://pandas.pydata.org/docs/user_guide/io.html?highlight=storage_options#reading-writing-remote-files>`.

`dtype_backend` : {'numpy_nullable', 'pyarrow'}, default 'numpy_nullable'

Back-end data type applied to the resultant `DataFrame` (still experimental). Behaviour is as follows:

* `"numpy_nullable"`: returns nullable-dtype-backed `DataFrame` (default).

* `"pyarrow"`: returns pyarrow-backed nullable `ArrowDtype` `DataFrame`.

.. versionadded:: 2.0

Returns

`DataFrame` or `TextFileReader`

A comma-separated values (csv) file is returned as two-dimensional data structure with labeled axes.

See Also

`DataFrame.to_csv` : Write `DataFrame` to a comma-separated values (csv) file.

`read_table` : Read general delimited file into `DataFrame`.

`read_fwf` : Read a table of fixed-width formatted lines into DataFrame.

Examples

```
>>> pd.read_csv('data.csv') # doctest: +SKIP
```

File: c:\users\admin\appdata\local\programs\python\python312\lib\site-packages\pandas\io\parsers\readers.py

Type: function

```
In [5]: #printing the first 5 rows of the dataset
diabetes_dataset.head()
```

```
Out[5]:
```

	gender	age	hypertension	heart_disease	smoking_history	bmi	HbA1c_level	blood_sugar
--	--------	-----	--------------	---------------	-----------------	-----	-------------	-------------

0	Female	80.0	0	1	never	25.19	6.6	1
---	--------	------	---	---	-------	-------	-----	---

1	Female	54.0	0	0	No Info	27.32	6.6	1
---	--------	------	---	---	---------	-------	-----	---

2	Male	28.0	0	0	never	27.32	5.7	1
---	------	------	---	---	-------	-------	-----	---

3	Female	36.0	0	0	current	23.45	5.0	1
---	--------	------	---	---	---------	-------	-----	---

4	Male	76.0	1	1	current	20.14	4.8	1
---	------	------	---	---	---------	-------	-----	---

◀  ▶

```
In [6]: # number of rows and Columns in this dataset
diabetes_dataset.shape
```

```
Out[6]: (100000, 9)
```

```
In [7]: #getting the statistical measures of the data
diabetes_dataset.describe()
```

```
Out[7]:
```

	age	hypertension	heart_disease	bmi	HbA1c_level	blood_sugar
--	-----	--------------	---------------	-----	-------------	-------------

count	100000.000000	100000.000000	100000.000000	100000.000000	100000.000000	100000.000000
-------	---------------	---------------	---------------	---------------	---------------	---------------

mean	41.885856	0.07485	0.039420	27.320767	5.527507	1.000000
------	-----------	---------	----------	-----------	----------	----------

std	22.516840	0.26315	0.194593	6.636783	1.070672	0.000000
-----	-----------	---------	----------	----------	----------	----------

min	0.080000	0.000000	0.000000	10.010000	3.500000	0.000000
-----	----------	----------	----------	-----------	----------	----------

25%	24.000000	0.000000	0.000000	23.630000	4.800000	0.000000
-----	-----------	----------	----------	-----------	----------	----------

50%	43.000000	0.000000	0.000000	27.320000	5.800000	0.000000
-----	-----------	----------	----------	-----------	----------	----------

75%	60.000000	0.000000	0.000000	29.580000	6.200000	0.000000
-----	-----------	----------	----------	-----------	----------	----------

max	80.000000	1.000000	1.000000	95.690000	9.000000	1.000000
-----	-----------	----------	----------	-----------	----------	----------

◀  ▶

```
In [8]: diabetes_dataset['diabetes'].value_counts()
```

```
Out[8]: diabetes
0    91500
1     8500
Name: count, dtype: int64
```

```
In [9]: diabetes_dataset.groupby('diabetes').mean(numeric_only=True)
```

Out[9]:

	age	hypertension	heart_disease	bmi	HbA1c_level	blood_glucose
diabetes						
0	40.115187	0.058984	0.029235	26.887163	5.396761	132.8
1	60.946588	0.245647	0.149059	31.988382	6.934953	194.0

```
In [10]: #separating the data and labels
x=diabetes_dataset.drop(columns='diabetes',axis=1)
y=diabetes_dataset['diabetes']

print("x shape:",x.shape)
print("y shape:",y.shape)
```

x shape: (100000, 8)

y shape: (100000,)

In [11]: print(x)

	gender	age	hypertension	heart_disease	smoking_history	bmi \
0	Female	80.0	0	1	never	25.19
1	Female	54.0	0	0	No Info	27.32
2	Male	28.0	0	0	never	27.32
3	Female	36.0	0	0	current	23.45
4	Male	76.0	1	1	current	20.14
...	...	...	...	...	...	...
99995	Female	80.0	0	0	No Info	27.32
99996	Female	2.0	0	0	No Info	17.37
99997	Male	66.0	0	0	former	27.83
99998	Female	24.0	0	0	never	35.42
99999	Female	57.0	0	0	current	22.43

	HbA1c_level	blood_glucose_level
0	6.6	140
1	6.6	80
2	5.7	158
3	5.0	155
4	4.8	155
...	...	...
99995	6.2	90
99996	6.5	100
99997	5.7	155
99998	4.0	100
99999	6.6	90

[100000 rows x 8 columns]

In [12]: print(y)

```

0      0
1      0
2      0
3      0
4      0
..
99995  0
99996  0
99997  0
99998  0
99999  0
Name: diabetes, Length: 100000, dtype: int64

```

Data Standardization

```

In [13]: from sklearn.preprocessing import LabelEncoder

le=LabelEncoder()
diabetes_dataset['smoking_history']=le.fit_transform(diabetes_dataset['smoking_h

In [14]: print(diabetes_dataset.columns)

Index(['gender', 'age', 'hypertension', 'heart_disease', 'smoking_history',
      'bmi', 'HbA1c_level', 'blood_glucose_level', 'diabetes'],
      dtype='object')

In [15]: from sklearn.preprocessing import LabelEncoder

In [16]: le=LabelEncoder()
diabetes_dataset['gender']=le.fit_transform(diabetes_dataset['smoking_history'])

In [17]: x=diabetes_dataset.drop(columns='diabetes',axis=1)
y=diabetes_dataset['diabetes']

In [18]: from sklearn.preprocessing import StandardScaler

scaler=StandardScaler()
x=scaler.fit_transform(x)

In [19]: from sklearn.preprocessing import LabelEncoder

le=LabelEncoder()

for col in diabetes_dataset.select_dtypes(include='object').columns:
    diabetes_dataset[col]=le.fit_transform(diabetes_dataset[col])

In [20]: x=diabetes_dataset.drop('diabetes',axis=1)
y=diabetes_dataset['diabetes']

from sklearn.preprocessing import StandardScaler
scaler=StandardScaler()
x=scaler.fit_transform(x)

scaler=StandardScaler()

In [21]: print(diabetes_dataset.head())

```

	gender	age	hypertension	heart_disease	smoking_history	bmi	\
0	4	80.0	0	1	4	25.19	
1	0	54.0	0	0	0	27.32	
2	4	28.0	0	0	4	27.32	
3	1	36.0	0	0	1	23.45	
4	1	76.0	1	1	1	20.14	

	HbA1c_level	blood_glucose_level	diabetes
0	6.6	140	0
1	6.6	80	0
2	5.7	158	0
3	5.0	155	0
4	4.8	155	0

```
In [22]: scaler.fit(x)
standardized_data=scaler.transform(x)
x=standardized_data
y=diabetes_dataset['diabetes']
print(x)
print(y)
```

```
[ [ 9.63326710e-01  1.69270354e+00 -2.84439447e-01 ... -3.21055792e-01
    1.00170572e+00  4.77042159e-02]
  [-1.15346777e+00  5.38006427e-01 -2.84439447e-01 ... -1.15583678e-04
    1.00170572e+00 -1.42620999e+00]
  [ 9.63326710e-01 -6.16690686e-01 -2.84439447e-01 ... -1.15583678e-04
    1.61108022e-01  4.89878478e-01]
  ...
  [ 4.34128089e-01  1.07094356e+00 -2.84439447e-01 ...  7.67292549e-02
    1.61108022e-01  4.16182767e-01]
  [ 9.63326710e-01 -7.94336396e-01 -2.84439447e-01 ...  1.22036126e+00
    -1.42668764e+00 -9.34905254e-01]
  [-6.24269153e-01  6.71240710e-01 -2.84439447e-01 ... -7.36921977e-01
    1.00170572e+00 -1.18055762e+00]]
0      0
1      0
2      0
3      0
4      0
..
99995  0
99996  0
99997  0
99998  0
99999  0
Name: diabetes, Length: 100000, dtype: int64
```

```
In [23]: standardized_data= scaler.transform(x)
print(standardized_data)
```

```
[ [ 9.63326710e-01  1.69270354e+00 -2.84439447e-01 ... -3.21055792e-01
    1.00170572e+00  4.77042159e-02]
 [-1.15346777e+00  5.38006427e-01 -2.84439447e-01 ... -1.15583678e-04
    1.00170572e+00 -1.42620999e+00]
 [ 9.63326710e-01 -6.16690686e-01 -2.84439447e-01 ... -1.15583678e-04
    1.61108022e-01  4.89878478e-01]
 ...
 [ 4.34128089e-01  1.07094356e+00 -2.84439447e-01 ...  7.67292549e-02
    1.61108022e-01  4.16182767e-01]
 [ 9.63326710e-01 -7.94336396e-01 -2.84439447e-01 ...  1.22036126e+00
    -1.42668764e+00 -9.34905254e-01]
 [-6.24269153e-01  6.71240710e-01 -2.84439447e-01 ... -7.36921977e-01
    1.00170572e+00 -1.18055762e+00]]
```

```
In [24]: x=standardized_data
         y=diabetes_dataset['diabetes']
         print(x)
         print(y)
```

```
[ [ 9.63326710e-01  1.69270354e+00 -2.84439447e-01 ... -3.21055792e-01
    1.00170572e+00  4.77042159e-02]
 [-1.15346777e+00  5.38006427e-01 -2.84439447e-01 ... -1.15583678e-04
    1.00170572e+00 -1.42620999e+00]
 [ 9.63326710e-01 -6.16690686e-01 -2.84439447e-01 ... -1.15583678e-04
    1.61108022e-01  4.89878478e-01]
 ...
 [ 4.34128089e-01  1.07094356e+00 -2.84439447e-01 ...  7.67292549e-02
    1.61108022e-01  4.16182767e-01]
 [ 9.63326710e-01 -7.94336396e-01 -2.84439447e-01 ...  1.22036126e+00
    -1.42668764e+00 -9.34905254e-01]
 [-6.24269153e-01  6.71240710e-01 -2.84439447e-01 ... -7.36921977e-01
    1.00170572e+00 -1.18055762e+00]]
```

```
0      0
1      0
2      0
3      0
4      0
```

```
..
```

```
99995   0
99996   0
99997   0
99998   0
99999   0
```

```
Name: diabetes, Length: 100000, dtype: int64
```

Train Test Split

```
In [25]: x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,stratify=y,rand
         print(x.shape,x_train.shape,x_test.shape)
```

```
(100000, 8) (80000, 8) (20000, 8)
```

Training The Model

```
In [26]: from sklearn import svm
```

```
In [27]: print(x_train.shape)
         print(y_train.shape)
```

```
(80000, 8)
(80000,)
```

```
In [31]: classifier=svm.SVC(kernel='linear')
```

```
In [32]: #Training the support vector Machine Classifier
classifier.fit(x_train,y_train)
```

```
Out[32]:
```

▼ SVC ⓘ ?

► Parameters

► Fitted attributes

Model Evaluation

Accuracy Score

```
In [ ]: from sklearn import svm

classifier=svm.SVC(kernel='linear')

classifier.fit(x_train,y_train)
```

```
In [37]: x_train_prediction=classifier.predict(x_train)

from sklearn.metrics import accuracy_score

training_data_accuracy=accuracy_score(y_train,x_train_prediction)
print("Accuracy Score of the training data:",training_data_accuracy)
```

Accuracy Score of the training data: 0.960475

```
In [38]: x_test_prediction=classifier.predict(x_test)
test_data_accuracy=accuracy_score(y_test,x_test_prediction)
print("Accuracy Score of the test data:",test_data_accuracy)
```

Accuracy Score of the test data: 0.9614

```
In [35]: #accuracy score on the training data
x_train_prediction=classifier.predict(x_train)
training_data_accuracy=accuracy_score(x_train_prediction,y_train)
print('Accuracy Score of the training data:',training_data_accuracy)
```

Accuracy Score of the training data: 0.960475

```
In [40]: #accuracy score on the test data
x_test_prediction=classifier.predict(x_test)
test_data_accuracy=accuracy_score(x_test_prediction,y_test)
print('Accuracy Score of the training data:',test_data_accuracy)
```

Accuracy Score of the training data: 0.9614

Making a Predictive System

```
In [43]: input_data=(54,0,0,70,54.7,6,100,0)
```

```
#change input_data to numpyarray
input_data_as_numpy_array=np.asarray(input_data)

#reshape the array as we are predicting for one instance
input_data_resaped=input_data_as_numpy_array.reshape(1,-1)

#standardize the input data
std_data=scaler.transform(input_data_resaped)
print(std_data)

prediction=classifier.predict(std_data)
print(prediction)

if(prediction[0]==0):
    print('The person is not diabetic')
else:
    print('The person is diabetic')
```

```
[[ 5.40000000e+01 -1.43387524e-16 -5.82645043e-17  7.00000000e+01
   5.47000000e+01  6.00000000e+00  1.00000000e+02  2.84217094e-16]]
[1]
The person is diabetic
```

In []: